Plan Maestro de Pruebas del Sistema

Software Test Documentation (STD)

Sistema de Facturación Electrónica Asistida por Inteligencia Artificial para la Gestión Tributaria en las MiPymes

Identificador: TP-SGT-v1.0

Versión: 1.0

Estándar aplicado: IEEE 829-2008

Preparado por: Álvaro José Mosquera Morales
Código: 1005894956

Director del proyecto: Jair Enrique Sanclemente Castro, M.Sc.

Programa académico: Ingeniería de Sistemas

Institución: Universidad Santiago de Cali (USC)
Facultad de Ingeniería

Cali, Colombia — 2026

Tabla de contenidos

Tabla de contenidos 2
1 Identificador del plan de pruebas 4
2 Referencias 4
3 Introducción 4
4 Ítems bajo prueba 5
5 Riesgos De Software 6
6 Características para probar 6
7 Características no probadas 8
8 Enfoque y estrategia de pruebas 8
8.1 Niveles de pruebas 8
8.2 Herramientas utilizadas 8
8.3 Gestión de configuración 10
8.4 Métricas recolectadas 10
8.5 Mecanismos de concurrencia y procesamiento asíncrono 10
9 Criterios de éxito y fracaso 10
10 Criterios de suspensión y reanudación 11
10.1 Criterios de suspensión 11
10.2 Criterios de reanudación 11
11 Entregables de pruebas 11
12 Tareas de pruebas pendientes 13
13 Ambiente de pruebas 13
13.1 Hardware 13
13.2 Software 13
13.3 Datos de prueba 13
14 Responsabilidades 14
15 Cronograma de pruebas 15
16 Riesgos de planificación y contingencias 15
17 Matriz de Pruebas Unitarias — Caja Blanca (TU) 15
18 Matriz de Pruebas Funcionales — Caja Negra (TF) 17
19 Matriz de Pruebas de Integración (TI) 17
20 Pruebas de Rendimiento (TR) 18
20.1 Metodología 18
20.2 Resultados 18
20.3 Análisis del cuello de botella 18
21 Pruebas de Estrés (TS) 19
22 Resumen de Resultados 20
23 Conclusiones y Recomendaciones 20

23.1 Conclusiones 20
23.2 Recomendaciones 20

1 Identificador del plan de pruebas

Este documento constituye el Plan Maestro de Pruebas del Sistema de Gestión Tributaria. Cubre los niveles de prueba unitaria (caja blanca), funcional (caja negra), integración, rendimiento y estrés. Su alcance corresponde a la versión 1.0 del sistema, desarrollada como proyecto de grado en la Universidad Santiago de Cali bajo el estándar IEEE 829-2008.

2 Referencias

SRS v1.0 — Requerimientos Funcionales y No funcionales del Sistema de Gestión Tributaria (ISO/IEC/IEEE 29148:2018).

UCS v1.0 — Documento de Casos de Uso (formato Alistair Cockburn).

IEEE 829-2008 — IEEE Standard for Software and System Test Documentation.

ISO/IEC/IEEE 29148:2018 — Systems and software engineering: Requirements engineering.

DIAN. Resolución 000042 de 2020. Requisitos técnicos para la factura electrónica bajo UBL 2.1.

MinHacienda. Decreto 358 de 2020. Obligatoriedad de la facturación electrónica.

Sommerville, I. (2005). Ingeniería del software. Pearson Educación.

Repositorio del sistema: https://github.com/AlvaroJMosquera/gestion-tributaria

3 Introducción

Este documento describe el plan maestro de pruebas del Sistema de Gestión Tributaria, una aplicación de escritorio en Python 3.13 que automatiza la extracción, persistencia y análisis de facturas electrónicas UBL 2.1 bajo el estándar DIAN Colombia, integrando un asistente de Inteligencia Artificial para consultas tributarias en lenguaje natural.

El sistema aborda una necesidad concreta de las MiPymes colombianas: la gestión manual de facturas electrónicas genera altos costos operativos, riesgo de error humano y limitada capacidad analítica tributaria. Las pruebas documentadas en este plan tienen como propósito verificar que el sistema cumple con los 18 requerimientos funcionales y 13 no funcionales especificados en el SRS v1.0, garantizando seguridad multi-tenant, integridad transaccional y rendimiento adecuado bajo condiciones de carga real.

El alcance del plan cubre las siguientes modalidades de prueba: unitarias (caja blanca sobre funciones críticas de seguridad y parsing), funcionales (caja negra desde la perspectiva del usuario), integración entre capas, rendimiento bajo volúmenes reales y estrés ante condiciones extremas. Las pruebas se ejecutaron con PyTest 8.1 sobre Python 3.13.5 en Windows 11.

4 Ítems bajo prueba

Los siguientes módulos del sistema fueron objeto de prueba en esta versión 1.0:

Módulo	Capa	Función principal	Versión
sql_assistant.py	Aplicación	Traducción NL→SQL + guardas de seguridad multi-tenant	1.0
processor.py	Aplicación	Motor ETL de facturas XML UBL 2.1 con tolerancia a errores	1.0
auth.py	Aplicación	Autenticación bcrypt + gestión de sesiones por tenant	1.0
models.py	Dominio	Entidades ORM tributarias (Tenant, Batch, Document, Line, Party, DocumentTax)	1.0
db_dedup.py	Dominio	Lógica de deduplicación por CUFE/CUDE con HashSet en memoria	1.0
db_repository.py	Infraestructura	Operaciones CRUD con SAVEPOINT por documento	1.0
db_config.py	Infraestructura	Pool de conexiones SQLAlchemy con pool_pre_ping=True	1.0
gui.py / login_window.py	Presentación	Interfaz Tkinter/ttkbootstrap con threading para UI no bloqueante	1.0

5 Riesgos De Software

Riesgo identificado	Probabilidad	Impacto	Mitigación aplicada
Inyección SQL a través de consultas generadas por LLM	Alta	Crítico	enforce_tenant_guards() sanitiza y filtra el 100 % de las consultas antes de ejecutarlas
Mezcla de datos entre organizaciones (multi-tenant)	Alta	Crítico	Variable de sesión PostgreSQL + filtros WHERE en todas las tablas con tenant_id
Corrupción del lote ante XML malformado	Alta	Alto	SAVEPOINT por documento + recover=True en lxml + log de errores.xlsx
Duplicados en base de datos por re-ejecución del lote	Media	Alto	Índices únicos CUFE/CUDE por tenant + lógica upsert_idempotent
Bloqueo de GUI durante procesamiento masivo	Media	Medio	threading.Thread + log_callback Observer para actualización asíncrona
Pérdida de conexión con Supabase durante carga	Media	Alto	pool_pre_ping=True detecta conexiones zombie y reconecta automáticamente
Variaciones de estructura en documentos XML reales	Alta	Medio	Pruebas con 1.358 documentos reales + 17 archivos corruptos controlados

6 Características para probar

Las siguientes características del sistema fueron verificadas durante las pruebas, organizadas por prioridad desde la perspectiva del usuario:

Característica	Prioridad	RF relacionado	Tipo de prueba
Autenticación segura con hashing bcrypt	Alta	RF-01, RF-16	TU, TF
Aislamiento de datos entre organizaciones (multi-tenant)	Crítica	RF-03, RF-13, RF-14	TU, TI
Procesamiento de facturas XML UBL 2.1 con tolerancia a errores	Alta	RF-04, RF-05, RF-06	TU, TF, TS
Deduplicación de documentos por CUFE/CUDE	Alta	RF-07, RF-11	TF, TI
Persistencia transaccional con SAVEPOINT por documento	Crítica	RF-10	TI, TS
Exportación de reportes XLSX con timestamp	Alta	RF-08	TF

Consulta en lenguaje natural con asistente IA	Media	RF-12, RF-13, RF-14, RF-15	TU, TF
Sanitización de SQL generado por LLM	Crítica	RF-13, RF-14	TU
Procesamiento en segundo plano sin bloqueo de GUI	Media	RF-17	TF
Reconexión automática ante pérdida de red	Media	RNF-08	TI
Rendimiento bajo carga masiva (1.358 documentos reales)	Alta	RNF-06	TR
Estabilidad ante archivos corruptos y concurrencia de tenants	Alta	RNF-07	TS

7 Características no probadas

Las siguientes áreas quedaron fuera del alcance de este plan de pruebas versión 1.0, con justificación explícita:

Característica no probada	Justificación
Integración directa con API de la DIAN	Durante el desarrollo no fue posible concretar un acuerdo con un proveedor tecnológico habilitado. Queda planteada como línea de investigación futura.
Pruebas en entorno Linux	Las pruebas de rendimiento y estrés se ejecutaron únicamente en Windows 11 / Python 3.13.5. La compatibilidad Linux (RNF-13) se verificó solo a nivel de instalación.
Pruebas de seguridad externas (penetration testing)	Fuera del alcance del proyecto de grado. El aislamiento multi-tenant fue verificado internamente mediante pruebas TU y TI.
Empaquetado y distribución con Inno Setup	El instalador .exe fue generado con PyInstaller + Inno Setup 6 pero no fue sometido a pruebas automatizadas formales.

8 Enfoque y estrategia de pruebas

8.1 Niveles de pruebas

La estrategia siguió un enfoque incremental: pruebas unitarias sobre funciones críticas aisladas, luego funcionales desde la perspectiva del usuario, integración entre capas, y finalmente rendimiento y estrés sobre datos reales. Este orden garantizó que cada nivel asumiera la solidez del nivel anterior, facilitando la localización precisa de fallos.

8.2 Herramientas utilizadas

Herramienta	Versión	Uso	Observación
PyTest	8.1	Framework principal	Descubrimiento automático, fixtures, parametrización
pytest-mock	3.14	Mocking	Mock de engine SQLAlchemy, Gemini, Ollama, callbacks de log
pytest-cov	5.0	Cobertura	Meta ≥ 80 % en módulos backend (TBD v1.1)
unittest.mock	Stdlib	Mocking	Parcheado de dependencias externas (BD, IA)
SQLAlchemy + SQLite in-memory	—	BD de pruebas	Repositorio sin Supabase para pruebas unitarias aisladas
lxml	5.1	Generación XML	XMLs UBL 2.1 sintéticos reproducibles para pruebas

			del parseo
Pandas / openpyxl	Stdlib	Validación Excel	Verificación de archivos XLSX generados en TF
test_rendimiento.py	Custom	Benchmark	Medición de rendimiento y estrés con XMLs sintéticos

8.3 Gestión de configuración

El código fuente del sistema se gestiona en el repositorio https://github.com/AlvaroJMosquera/gestion-tributaria. Los scripts de prueba están en el directorio tests/. Los resultados de las pruebas de rendimiento son reproducibles ejecutando: python tests/test_rendimiento.py en Python 3.13.5 / Windows 11.

8.4 Métricas recolectadas

- Pruebas unitarias: número de casos PASSED / FAILED y porcentaje de aprobación.
- Pruebas funcionales: número de escenarios exitosos sobre el total ejecutado.
- Pruebas de integración: estado Completado / Fallido por escenario.
- Pruebas de rendimiento: tiempo total, desglose por fase (parseo / DataFrame / Excel / upload BD), filas generadas y tamaño del archivo.
- Pruebas de estrés: tiempo de ejecución, tasa XMLs/seg, registros procesados y estabilidad del sistema.

8.5 Mecanismos de concurrencia y procesamiento asíncrono

El sistema implementa un esquema de concurrencia basado en threading.Thread para separar el hilo de procesamiento ETL del hilo principal de la interfaz gráfica. La comunicación entre hilos se realiza mediante queue.Queue, que actúa como canal seguro (thread-safe) para transmitir mensajes de progreso desde el procesador hacia el panel de log en ttkbootstrap, garantizando que la barra de progreso y los mensajes se actualicen en tiempo real sin bloquear la GUI (RNF-05). El hilo de inferencia IA bloquea IO durante la espera de respuesta del LLM, pero el hilo principal de orquestación nunca se congela gracias a esta separación.

Para el procesamiento de archivos ZIP, el sistema utiliza zipfile.ZipFile.open(name) con lectura en modo stream/buffer, evitando descomprimir los archivos en el filesystem de forma permanente. Esto significa que los XMLs dentro de los ZIPs se leen directamente como flujos de bytes en memoria sin materializarse en disco, lo que permite manejar archivos de gran tamaño e incluso ZIP Bombs (TS-07) sin agotamiento de memoria. El costo amortizado de memoria es O(1) por factura al ser procesadas individualmente, no acumulando todo el lote en RAM simultáneamente.

9 Criterios de éxito y fracaso

Nivel de prueba	Criterio de éxito (PASS)	Criterio de fracaso (FAIL)
Unitarias (TU)	100 % de casos PASSED. Cero fallos en funciones de seguridad (TU-03 a TU-10).	Cualquier fallo en guardas de seguridad o parsing XML.
Funcionales (TF)	100 % de escenarios exitosos. GUI responde durante procesamiento.	Cualquier escenario fallido o bloqueo de interfaz.

Integración (TI)	Todos los escenarios Completado. Aislamiento multi-tenant verificado.	Mezcla de datos entre tenants o fallo ETL completo.
Rendimiento (TR)	50 XMLs en < 1s (solo Excel). 1.358 XMLs en < 30 min con BD.	Superación de umbrales definidos en RNF-06.
Estrés (TS)	Sistema estable ante XMLs corruptos y concurrencia de 3 tenants. Sin mezcla de datos.	Caída del sistema, mezcla de datos o agotamiento de memoria.

10 Criterios de suspensión y reanudación

10.1 Criterios de suspensión

- Fallo en cualquier prueba de aislamiento multi-tenant (TU-03 a TU-08, TI-03): suspensión inmediata hasta corrección del módulo enforce_tenant_guards().
- Fallo en pruebas de bloqueo DELETE/UPDATE (TU-09, TU-10): suspensión de pruebas funcionales y de integración hasta resolución.
- Corrupción de datos en base de datos durante pruebas de integración: suspensión y restauración del esquema desde init_db.py.
- Más del 30 % de casos funcionales fallidos en una sesión: suspensión para revisión de la build.

10.2 Criterios de reanudación

- Corrección documentada del defecto en el repositorio con commit identificado.
- Re-ejecución exitosa de los casos que originaron la suspensión antes de continuar con el resto de la suite.
- Restauración verificada del ambiente de pruebas (base de datos vacía, Ollama disponible o mockeado

11 Entregables de pruebas

- STD v1.0 — Este documento (Plan Maestro de Pruebas IEEE 829).
- Resultados pytest -v — Salida de consola con 14/14 Completadas de las pruebas unitarias.
- errores.xlsx — Archivo generado automáticamente por el sistema con el listado de XMLs no procesables durante las pruebas de estrés TS-04 y TS-06.
- test_rendimiento.py — Script reproducible para pruebas de rendimiento y estrés con XMLs sintéticos UBL 2.1.
- Repositorio de pruebas — Directorio tests/ en https://github.com/AlvaroJMosquera/gestion-tributaria con todos los scripts de prueba.
- Nota: Los 1.358 documentos XML reales utilizados en TR-03 son documentos fiscales reales de una MiPyme colombiana y no pueden publicarse por razones

de protección de datos personales (Ley 1581 de 2012). Su existencia y procesamiento exitoso queda evidenciada en los resultados documentados en la Sección 16.

12 Tareas de pruebas pendientes

Tarea	Versión objetivo	Justificación
Pruebas de rendimiento en entorno Linux (Ubuntu 22.04)	v1.1	RNF-13 verificado solo en Windows 11 en esta versión.
Pruebas de aceptación con usuarios reales (contadores MiPyme)	v2.0	Requiere acuerdo con organizaciones participantes.
Pruebas de integración con proveedor tecnológico habilitado DIAN	v2.0	TBD-01 del SRS. Descarga automática de facturas pendiente.

13 Ambiente de pruebas

13.1 Hardware

- Computador de escritorio / portátil con Windows 11 (64-bit).
- Memoria RAM: mínimo 8 GB para pruebas con lotes de 1.358 documentos.
- Almacenamiento: 2 GB disponibles para archivos XML de prueba y base de datos SQLite in-memory.

13.2 Software

- Python 3.13.5 con dependencias: lxml 5.1, SQLAlchemy 2.x, Polars, Pandas, xlsxwriter, ttkbootstrap, python-dotenv.
- PyTest 8.1+ pytest-mock 3.14+ pytest-cov 5.0.
- Ollama con modelo Llama 3.1 (8B) instalado localmente para pruebas TF-10 con asistente IA.
- Cuenta Supabase activa con esquema inicializado mediante init_db.py para pruebas de integración TI-01 a TI-04.
- SQLite in-memory (incluido en Python stdlib) para pruebas unitarias TU sin dependencia de red.

13.3 Datos de prueba

- 1.358 documentos XML reales bajo estándar UBL 2.1 DIAN — utilizados en TR-03, TS-01.
- XMLs sintéticos generados por test_rendimiento.py — utilizados en TR-01, TR-02, TS-02, TS-03.
- 17 archivos XML corruptos con caracteres inválidos (\x00–\x08) — utilizados en TS-04, TF-09.
- Lote mixto: 10 XMLs corruptos + 40 válidos — utilizado en TS-06.
- 3 tenants simulados con 50, 80 y 60 XMLs respectivamente — utilizados en TS-05.

14 Responsabilidades

Actividad	Responsable	Rol
Diseño y ejecución de pruebas unitarias y funcionales	Álvaro José Mosquera Morales	Desarrollador / Tester
Diseño y ejecución de pruebas de integración	Álvaro José Mosquera Morales	Desarrollador / Tester
Diseño y ejecución de pruebas de rendimiento y estrés	Álvaro José Mosquera Morales	Desarrollador / Tester
Revisión y aprobación del plan de pruebas	Jair Enrique Sanclemente Castro, M.Sc.	Director del proyecto
Verificación de cumplimiento normativo (ISO 29148, DIAN)	Jair Enrique Sanclemente Castro, M.Sc.	Director del proyecto
Aprobación final del documento STD v1.0	Jair Enrique Sanclemente Castro, M.Sc.	Director del proyecto

15 Cronograma de pruebas

Actividad	Fase del cascada	Período	Estado
Pruebas unitarias (TU-01 a TU-14)	Fase 3 — Implementación	Enero–febrero 2026	Completado
Pruebas funcionales (TF-01 a TF-10)	Fase 3 — Implementación	Febrero 2026	Completado
Pruebas de integración (TI-01 a TI-05)	Fase 4 — Integración	Febrero–marzo 2026	Completado
Pruebas de rendimiento (TR-01 a TR-04)	Fase 5 — Operación	Marzo 2026	Completado
Pruebas de estrés (TS-01 a TS-08)	Fase 5 — Operación	Marzo 2026	Completado
Documentación STD v1.0	Post-desarrollo	Marzo 2026	Completado

16 Riesgos de planificación y contingencias

- Indisponibilidad de Supabase durante pruebas de integración: mitigación con SQLite in-memory para pruebas unitarias y mock de conexión para pruebas funcionales.
- Variaciones en el formato de XMLs reales no contempladas en pruebas sintéticas: mitigación ejecutando TR-03 sobre 1.358 documentos reales con 17 archivos corruptos controlados (TS-04).
- Latencia de red variable con Supabase: los umbrales de rendimiento (RNF-06: ≤ 2s/registro) se definieron considerando condiciones normales de red (latencia ≤ 100 ms). En TR-03 se obtuvo 0.90s/doc promedio.
- Cambios en la API de Gemini 2.5 Flash: el asistente IA tiene compatibilidad dual con Ollama local, garantizando que las pruebas TF-10 puedan ejecutarse sin conectividad.

17 Matriz de Pruebas Unitarias — Caja Blanca (TU)

Resultado global: 14/14 PASSED (100 %). Pruebas ejecutadas con pytest -v sobre el módulo sql_assistant.py y el motor de parsing XML. Framework: PyTest 8.1 con unittest.mock para aislar dependencias externas.

ID	Caso de prueba — Camino de control verificado	Resultado	RF relacionado
TU-01	Limpieza de SQL generado por IA: eliminación de fences Markdown (```sql)	PASSED	RF-13
TU-02	Validación y sanitización de punto y coma al final de la sentencia SQL	PASSED	RF-13
TU-03	Inyección de filtro tenant_id en tabla documents (rama: sin WHERE previo)	PASSED	RF-03, RF-13

TU-04	Prevención de duplicidad de cláusulas WHERE (rama: WHERE existente)	PASSED	RF-03, RF-13
TU-05	Aplicación de guardas de seguridad en JOINs complejos con aliases dinámicos	PASSED	RF-13, RF-14
TU-06	Reescritura de consultas con funciones de agregación (GROUP BY, COUNT, SUM)	PASSED	RF-13
TU-07	Protección de datos en tabla de terceros (parties) con alias detectado	PASSED	RF-13
TU-08	Aislamiento de lotes de carga (batches) mediante filtro de tenant	PASSED	RF-03
TU-09	Bloqueo de operaciones DELETE fuera del tenant activo (regex específico)	PASSED	RF-14
TU-10	Bloqueo de operaciones UPDATE fuera del tenant activo (regex específico)	PASSED	RF-14
TU-11	Recuperación de errores en parsing XML: lxml recover=True + limpieza \x00–\x08	PASSED	RF-05
TU-12	Extracción completa de campos fiscales desde XML UBL 2.1 y Notas de Crédito: CUFE, CUDE, IVA, ICA, partes involucradas. Método: FacturaProcessor.procesar_xml_text + .extraer_lineas. Datos de entrada: Mock XML local para no exponer facturas reales.	APROBADO	RF-06
TU-13	Tolerancia a fallo ante XML con caracteres nulos (\x00) o estructura dañada. El módulo usa lxml con recover=True extrayendo partes inteligibles y descartando el resto como registro inválido, sin colapso de la aplicación.	APROBADO	RF-05
TU-14	Mockeo de llamadas HTTP a google.genai y API local de Ollama para verificar que la traducción NL→SQL inyecta siempre blindajes de tenant_id y RLS. Verifica que el flujo fuerza WHERE tenant_id = '...' ante cualquier consulta generada.	APROBADO	RF-13, RF-03

18 Matriz de Pruebas Funcionales — Caja Negra (TF)

Las pruebas funcionales validan el comportamiento del sistema desde la perspectiva del usuario, sin conocimiento de la implementación interna. Diseñadas usando partición de equivalencia y análisis de valores límite. Resultado global: 10/10 exitosas (100 %).

ID	Caso de prueba	Estado	Observaciones técnicas
TF-01	Login de usuario válido	Exitoso	Acceso concedido, tenant_id cargado en contexto de sesión. RF-01, RF-16.
TF-02	Login con contraseña incorrecta	Exitoso	Sistema retorna None; botón habilitado nuevamente. RF-16.
TF-03	Login con campos vacíos	Exitoso	Mensaje de advertencia sin consulta a BD. RF-16.
TF-04	Login con email en mayúsculas	Exitoso	Sistema normaliza a lowercase correctamente. RF-01.
TF-05	Selección de carpeta de entrada	Exitoso	Explorador de archivos abre y actualiza ruta en GUI. RF-04.
TF-06	Procesar solo Excel (sin BD)	Exitoso	Excel generado con timestamp; sin conexión a Supabase. RF-08.
TF-07	Procesar Excel + BD	Exitoso	Persistencia exitosa en Supabase con Batch creado. RF-10.
TF-08	Visualización de log en tiempo real	Exitoso	Mensajes de progreso actualizados sin bloqueo de GUI. RF-17, RNF-05.
TF-09	Manejo de XML corrupto/inválido	Exitoso	safe_parse() limpia \x00–\x07; fallo registrado en errores.xlsx; proceso no detenido. 4 de 5 variantes parseadas con recover; 1 excepción aislada. RF-05, RF-09.
TF-10	Consulta con Asistente IA	Exitoso	LLM generó SQL válido; enforce_tenant_guards() inyectó tenant_id; resultado devuelto en panel de chat. RF-12, RF-13, RF-15.

19 Matriz de Pruebas de Integración (TI)

Las pruebas de integración verifican la interacción correcta entre las capas del sistema (GUI → Procesador → Repositorio → Base de datos). Resultado global: 5/5 escenarios completados.

ID	Escenario	Estado	Componentes verificados	RF/RNF
TI-01	Procesamiento completo ETL + Excel + BD	Completado	GUI → Processor → Repository → Supabase	RF-10, RF-08
TI-02	Idempotencia de carga (mismo lote dos veces)	Completado	Índices únicos CUFE/CUDE + lógica upsert_idempotent	RF-11, RF-07

TI-03	Aislamiento multi-tenant (Tenant A no ve datos de Tenant B)	Completado	enforce_tenant_guards() + variable de sesión PostgreSQL	RF-03, RF-13, RF-14
TI-04	Reconexión automática tras pérdida de red	Completado	pool_pre_ping=True detecta y reconecta transparentemente	RNF-08
TI-05	Atomicidad de inserción en entorno transaccional aislado (Supabase/PostgreSQL). Verifica begin_nested() + rollback() ante IntegrityError sin detener la cola masiva. Idempotencia: mismo CUFE detectado por índice rápido, retorna sin error.	Completado	db_dedup.py: upsert_document_with_lines_idempotent	RF-10, RF-11

20 Pruebas de Rendimiento (TR)

20.1 Metodología

Las pruebas de rendimiento se ejecutaron bajo dos enfoques complementarios: (1) ambiente real con 1.358 documentos XML reales (TR-03), midiendo tiempos reales de parseo, consolidación y persistencia en Supabase; (2) XMLs sintéticos generados en memoria por test_rendimiento.py que replican fielmente la estructura UBL 2.1 del estándar DIAN, produciendo resultados reproducibles. Ambiente de ejecución: Python 3.13.5 / Windows 11.

20.2 Resultados

ID	Escenario de carga	Tiempo total	Desglose técnico
TR-01	50 XMLs UBL 2.1 (sintéticos)	0.33 s	Parseo: 0.01s | DataFrame: 0.003s | Excel: 0.32s | 150 filas | 17.8 KB
TR-02	200 XMLs UBL 2.1 (sintéticos)	0.13 s	Parseo: 0.03s | DataFrame: 0.001s | Excel: 0.10s | 600 filas | 54.8 KB
TR-03	1.358 XMLs reales (ambiente productivo)	1.636.58 s	Parseo: 163.66s | Upload BD: 1.227.43s | Excel: 245.49s | 8.780 registros
TR-04	Proyección: 5.000 XMLs	~6.024 s	Basado en tasa real: 0.83 docs/seg en Supabase. Cuello de botella: latencia de red.

20.3 Análisis del cuello de botella

En TR-03, el cuello de botella es la capa de persistencia en Supabase, que consumió el 75 % del tiempo total (1.227 de 1.636 segundos). El motor de parsing Python/lxml procesó los mismos 1.358 documentos en apenas 163 segundos. La capa de aplicación consolidó 8.780 registros en memoria en 13 segundos (TS-01). Esto confirma que la arquitectura de procesamiento es eficiente y que el rendimiento en producción está ligado a la latencia de red

hacia el servidor PostgreSQL remoto, no a la lógica de la aplicación. La optimización prioritaria para v2.0 es la implementación de bulk inserts, que reduciría la latencia de 0.90 s/doc a ~0.05 s/doc estimado.

21 Pruebas de Estrés (TS)

ID	Escenario	Resultado	Observación técnica
TS-01	8.780 registros consolidados en memoria (1.358 docs reales)	13 s — PASS	Consolidación Polars/Pandas en capa de aplicación; sin agotamiento de memoria.
TS-02	ZIP con 150 XMLs anidados en subcarpetas (0.25 MB)	0.08 s — PASS	1.893 filas extraídas; estructura recursiva procesada sin interrupciones. RF-04.
TS-03	500 XMLs consecutivos en lote único	0.08 s — PASS	6.367 XMLs/seg; 1.500 filas extraídas; sistema estable.
TS-04	17 archivos XML corruptos en lote real	Estable — PASS	17 fallos logueados en errores.xlsx; proceso no detenido; lote completado al 100 %. RF-09.
TS-05	Concurrencia: 3 tenants procesando simultáneamente	0.03 s — PASS	570 filas procesadas (50+80+60 XMLs); datos aislados por tenant; sin bloqueos ni mezcla. RF-03.
TS-06	10 XMLs corruptos mezclados con 40 válidos	0.005 s — PASS	Aislamiento confirmado: 10 errores registrados; 150 filas extraídas de XMLs válidos sin contaminación.
TS-07	Zip Bomb — archivo .zip comprimido con millones de líneas ficticias en XML. Intento de saturar memoria con entrada maliciosa.	Tolerante — PASS	zipfile.ZipFile.open(name) lee por streams/buffer sin cargar el XML completo en RAM plana. safe_parse() captura el colapso mediante límites de buffer. Garbage Collector de Python gestiona la memoria sin agotamiento.
TS-08	Desconexión violenta de red durante exportación activa a PostgreSQL (simulación de timeout en POST a Supabase Cloud).	Tolerante — PASS	SQLAlchemy atrapa OperationalError / SQLAlchemyError. El documento actual se detiene y la factura fallida se registra en el log. Al restablecer la conexión, los archivos remanentes se pueden subir; los ya persistidos están protegidos por idempotencia (CUFE ya existente).

22 Resumen de Resultados

Tipo de prueba	Total	Completadas	Fallidas	Tasa éxito	Estado
Unitarias (TU)	14	14	0	100 %	PASS
Funcionales (TF)	10	10	0	100 %	PASS
Integración (TI)	5	5	0	100 %	PASS
Rendimiento (TR)	4	4	0	100 %	PASS
Estrés (TS)	8	8	0	100 %	PASS
TOTAL	41	41	0	100 %	PASS

23 Conclusiones y Recomendaciones

23.1 Conclusiones

- El sistema cumple con el 100 % de los 18 requerimientos funcionales establecidos bajo ISO/IEC/IEEE 29148:2018, con evidencia trazable en cada caso de prueba documentado.
- La seguridad multi-tenant está garantizada en tres niveles independientes: variable de sesión PostgreSQL, inyección dinámica en cláusulas WHERE mediante enforce_tenant_guards(), y SAVEPOINTS de transacción por documento.
- La arquitectura en 4 capas demostró su utilidad práctica: el motor de parsing, la capa de base de datos y la interfaz gráfica pudieron modificarse independientemente durante el desarrollo sin afectar las demás capas.
- El sistema procesó exitosamente 1.358 documentos XML reales, persistió 8.780 registros y aisló 17 errores sin interrumpir el lote, demostrando robustez ante condiciones reales de producción.
- La idempotencia de carga fue verificada experimentalmente: ejecutar el mismo lote dos veces no genera duplicados ni errores en la base de datos (TI-02).
- El asistente IA demostró operación segura: enforce_tenant_guards() inyectó filtros de tenant en el 100 % de las tablas con tenant_id, bloqueando toda operación DELETE y UPDATE generada dinámicamente.
- El motor de parsing Python/lxml procesó hasta 6.367 XMLs/segundo en pruebas sintéticas. El cuello de botella del sistema es la latencia de red hacia Supabase (0.90 s/doc promedio), no la lógica de la aplicación.

23.2 Recomendaciones

- Migrar la capa de presentación de Tkinter a FastAPI + React para acceso remoto y pruebas de usabilidad con usuarios reales.
- Ejecutar pruebas de aceptación con contadores MiPyme reales para validar la usabilidad del asistente IA en contextos tributarios cotidianos.